Dynamic Speculators

Project Working Record [1]

Abstract

This document records ongoing experiments with a conditional generator that produces LoRA factors for an EAGLE3 drafter from sets of target hidden-state summaries. It details the set encoder, shared decoder, and full-factor output heads, and records fixed-update data scaling and the decision to retire LoRA-bank warmup. It describes the training and serving interfaces, a supplied four-model throughput comparison on in-distribution and out-of-distribution workloads, activation-cache measurements, and an unresolved output-repeatability investigation. A fixed-batch shared-adapter serving implementation is recorded separately from its pending GPU measurements. Every experiment states its available evidence, execution status, and missing information. The document is extended on request and does not present final conclusions.

1. Research Objectives and Questions

We investigate a generator that maps a set of prompt summaries to one shared adapter for a frozen EAGLE3 drafter. The current comparison comprises the original drafter, a trained static LoRA, and two generator checkpoints described as trained on 24K examples. Older benchmark models are outside this serving comparison. Historical training ablations are recorded separately: fixed-update data scaling and the decision to discontinue bank-based generator warmup.

The working questions are:

1. How sensitive is generator validation behavior to training-pool size under a fixed optimizer-update budget?
2. How do generator checkpoints behave relative to the original drafter and a static trained adapter under a specified serving workload?
3. How do recorded measurements differ between Nemotron validation prompts and selected Red Hat benchmark categories?
4. What changes when one generated adapter is shared by a fixed batch of requests at different concurrencies?
5. What time and memory costs arise from obtaining and reusing target activations during training?

These questions remain open. A numerical table records an observation under its measurement conditions; it does not establish general superiority, statistical significance, lossless acceleration, or completion of a planned experiment.

1.1. Record Conventions

Experiment identifiers remain stable as evidence is appended. We use *planned*, *implemented*, *partially measured*, *measured*, and *investigation deferred* as execution statuses. Evidence quality is stated separately: a measured value may still have incomplete provenance or no repetition. Dates distinguish receipt of evidence from execution when the latter is unknown. This version includes evidence available through 2026-10-23.

2. Method and System Description

2.1. Set-Conditioned Adapter Generation

The generator receives one summary per conditioning prompt, together with its prompt length. Current serving uses the hidden states at the last prompt token from the configured target layers. Prompt conditioning excludes response tokens. For a conditioning set $S = \{(h_i, \ell_i)\}_{i=1}^{n}$, a shared encoder produces representations z_i. The condition is constructed from their mean, population standard deviation, and the logarithm of the set size:

$$c = F([\mathrm{mean}_i\, z_i,\ \mathrm{std}_i\, z_i,\ \log(1+n)])\,. \tag{1}$$

Module and layer descriptors are combined with c before decoding full LoRA factors. For each adapted projection p, the resulting weight is

$$W_p' = W_p + \frac{\alpha}{r} B_p A_p. \tag{2}$$

[1] Internal experimental record, version 02. Correspondence to: Project record <repository issues>.

Preprint. October 23, 2026.

The current implementation adapts the drafter's V and output projections. The set produces one adapter shared by its examples, rather than independent adapters for each example. Set conditioning therefore differs from the number of independently scheduled inference requests.

2.2. Generator Architecture

The generator is a full-factor hypernetwork: its outputs are the adapter's actual matrices, rather than indices into a stored adapter bank. For a prompt summary of dimension d_h, the shared example encoder takes $[h_i, \log(1+\ell_i)]$ through a $(d_h+1)\to 512$ linear layer, GELU, a $512\to 128$ layer, and LayerNorm. Pooling the 128-dimensional mean and population standard deviation with $\log(1+n)$ gives 257 features. A $257 \to 128 \to 64$ MLP with GELU produces the condition. Pooling makes the result invariant to the ordering of conditioning examples; the count feature distinguishes different set sizes.

Each projection has a learned 32-dimensional module descriptor, alongside a 32-dimensional layer descriptor. In the current single-layer drafter the layer embedding has one entry. Concatenating both with the condition gives 128 features. A shared linear layer and GELU expand these to width 512, followed by two residual MLP blocks. Each block applies LayerNorm, linear, GELU, and linear before adding its input back.

Separate linear output heads for each projection produce $A_p \in \mathbb{R}^{r\times d_{\text{in},p}}$ and $B_p \in \mathbb{R}^{d_{\text{out},p}\times r}$. The rank-32 generator used in the recorded logs has approximately 293 million parameters; the large full-factor heads dominate its size. The bottleneck condition is shared, while the factors depend on the projection descriptor. Initially the head weights are zero, the A biases receive Kaiming initialization, and the B biases are zero, so the initial adapter update is zero.

This structure gives one representative LoRA for a conditioning set. An ordinary LoRA baseline instead learns one persistent pair of factors per projection. The generator's larger trainable capacity is a confound when comparing these approaches; matching rank alone does not match trainable parameter count. Architecture dimensions above follow the inspected implementation defaults; per-run configuration artifacts are needed to confirm historical overrides.

2.3. Training Interface

The target and base drafter remain frozen while generator parameters are optimized through generated factors. The current recipe uses `lk_hybrid`, rank 32, and alpha 64. Conditioning-set sizes are sampled from 1–8; four episodes are accumulated per optimizer update in the documented recipe. Token-budget microbatches share the episode's factors. Context-4 validation is a separate evaluation setting. Gradients from the drafter loss flow through the generated A/B factors into the encoder, decoder, and output heads. Thus the generator learns adapters that serve the drafting objective directly, without requiring a previously trained adapter as a target.

Training replays saved prompt/response token trajectories through an external vLLM activation extractor. Only activations needed by the upcoming work are constructed, and CPU RAM retains reusable tensors within a bounded cache. The training pipeline does not require downloading the full activation dump. These describe the implementation and recipe, not verified configurations of every historical measurement.

2.4. Serial and Fixed-Batch Serving

Serial serving conditions one adapter on one prompt. The new fixed-batch mode collects a closed group, obtains all prompt summaries from full target prefill, generates one adapter, and merges it into the shared drafter before draft prefill. The weights remain fixed until all group members finish or are aborted. New requests cannot replace members that finish early. A subsequent group receives its own adapter after the previous group ends.

The configured concurrency is a capacity, not a guarantee of actual batch size. A collection timeout permits partial groups, including the final workload tail. Original-drafter and static-LoRA baselines use the same closed admission policy in the new sweep. Static LoRA uses a permanent merge. No GPU throughput or acceptance result for this batching policy is included in this version.

3. Shared Experimental Setup

3.1. Models and Data

The documented model family is Qwen3-8B with the Red Hat EAGLE3 drafter. The current suite compares the original drafter, a trained ordinary LoRA, and generator snapshots labeled 2,000 and 4,000 steps. Generator data size is reported as 24K examples. Exact training progress must ultimately come from checkpoint metadata; labels are retained without treating them as verification.

Nemotron math, code, STEM, and chat are designated in-distribution evaluation categories. Red Hat HumanEval, math reasoning, and summarization are designated out-of-distribution categories in this comparison. These labels describe the chosen evaluation sources; they do not by themselves establish absence of all semantic overlap with training. Validation rows are excluded from optimization by the documented trajectory membership contract.

The full-data workflow schedules six training subsets per

domain, totaling 24K examples, and supports ten planned epochs. It saves Hub snapshots every 1,000 optimizer updates and at completion, with context-4 validation every 500 updates in the current recipe. The schedule is not evidence that ten epochs have completed. This report includes only the supplied checkpoint observations.

3.2. Metrics and Measurement Boundaries

We keep the following quantities distinct:

- **Training and validation losses:** optimization/evaluation values under a specified conditioning and microbatch policy.
- **Offline agreement:** drafter/target argmax agreement in offline evaluation, not measured speculative acceptance.
- **Serving acceptance:** native speculative-decoding counters during actual generation; absent from the supplied current suite summary.
- **Throughput:** aggregate generated output tokens divided by measured wall seconds, including prefill and serving overhead.
- **Latency and systems timings:** request latency, first-token time, and stage wall time, when their artifacts are available.

3.3. Recipe Versus Verified Run Metadata

The independent serving implementation fixes BF16, eager execution, the V1 runner, and a pinned vLLM 0.31.0 integration. Suite defaults specify a seed, sample limit, output budget, and warmup count. These defaults must not be substituted for missing metadata from the supplied throughput summary. For that summary, hardware, concurrency, sample counts, seed, workload hashes, and exact checkpoint identities remain unverified. The cache pilot separately reports an A100 80 GB. Neither source establishes repeated-run uncertainty.

4. Experiments and Recorded Measurements

4.1. GEN-001: Data Scaling at Fixed Optimizer Budget

Status: Preliminary; single-seed visual evidence.

Evidence: W&B screenshots, including the comparison with ordinary LoRA supplied 2026-10-23. Raw metric exports and immutable run configurations are unavailable.

Question. Does more training data improve validation agreement within the same number of optimizer updates?

Setup. The direct generator arms use nested pools of 4K, 8K, and 16K examples: one, two, or four subsets per domain across math, code, STEM, and chat. All three run for 1,500 updates with seed 42. The documented recipe uses `lk_hybrid`, four episodes per update, training context sizes 1–8, rank 32, alpha 64, and an 8,192-token microbatch budget. AdamW uses learning rate 10^{-4}, weight decay 0.01, gradient clipping 0.85, and 100 learning-rate warmup updates. This scheduler warmup is distinct from bank reconstruction. Validation uses 100 held-out rows per domain from subset 00000, grouped at context size 4, every 250 updates including updates 0 and 1,500. The additional curve is labeled `scale-1-lora-gseed42`; its intended comparison is ordinary LoRA trained on the 4K pool at the same update budget. Historical configuration artifacts are needed to verify all matched settings.

Observed behavior. In Figure 1, generator validation curves largely overlap across data scales. Most improvement occurs early, with smaller gains thereafter. The ordinary LoRA curve improves more slowly and finishes visibly below the generators on math, code, and STEM. Chat differences are smaller, with a late dip in the scale-1 generator curve. Training loss is noisy, while the generator trends finish below the ordinary LoRA trend.

Working interpretation. On this split, seed, and update budget, pool size has a small observed effect compared with the gain from training itself. Increasing available data fourfold does not produce a correspondingly large validation gain in these curves. This is a fixed-update comparison: it neither matches epochs nor guarantees equal processed tokens, compute, or wall time. Larger pools receive less average reuse. The result therefore motivates asking whether longer training or broader validation would reveal additional data benefits, rather than concluding that data quantity is generally unimportant.

Limits. Validation rows are held out, but their subset identity is represented in every training pool. One seed and screenshots cannot establish statistical equivalence or a precise ranking. Generator versus LoRA differences also include the much larger generator parameter count. These training curves do not add an older generator row to the current serving benchmark tables.

4.2. GEN-004: Bank Warmup and the Direct-Training Decision

Status: Preliminary adaptation observation; bank workflow retired.

Evidence: Adaptation screenshot supplied 2026-10-23; historical protocol recorded in the 2026-10-07 working notes. Exact execution date and raw exports unavailable.

Question. Does reconstructing previously trained bank LoRAs provide a useful lasting advantage over training a fresh generator on the original data?

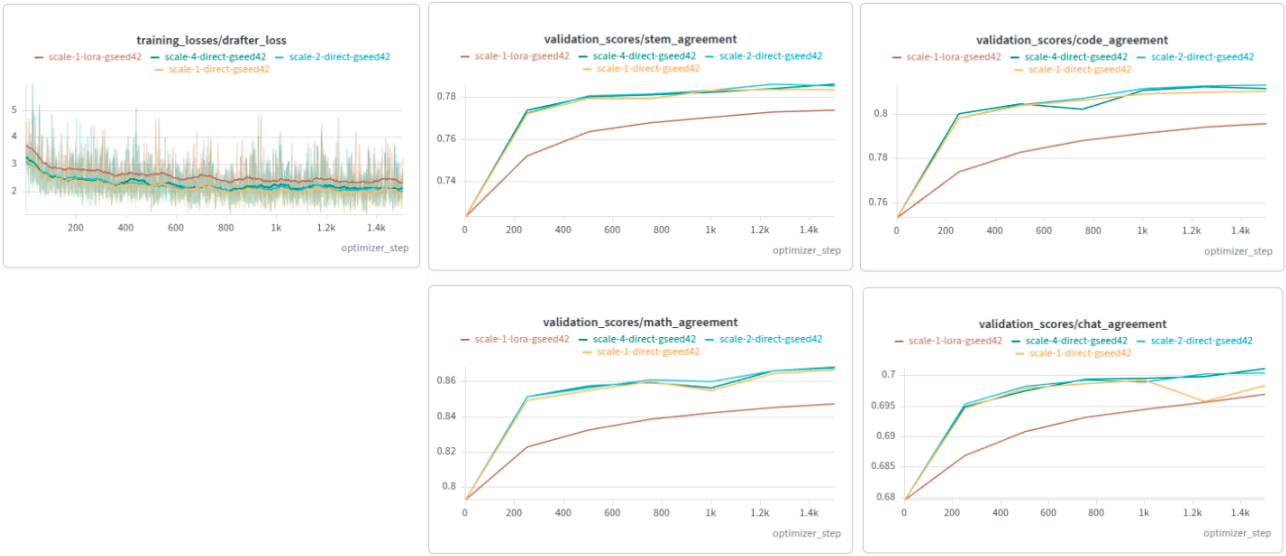


Figure 1. GEN-001: supplied training-loss and domain-validation plots. The three direct generator arms use 4K, 8K, and 16K training examples with the same 1,500-update budget. Ordinary scale-1 LoRA is included as a training baseline. Agreement is an offline validation metric, not measured serving acceptance. Curves are visual evidence; no raw values or uncertainty estimates are supplied.

Setup. The historical study adapts four initializations on `math-00001`: a bank-pretrained generator, a fresh generator, a transferred ordinary LoRA, and a fresh ordinary LoRA. The recorded protocol uses seed 42, last-token conditioning, training context sizes 1–8, and one adaptation epoch. The supplied plot spans roughly 200 optimizer updates and evaluates context 8; this is distinct from GEN-001's context-4 validation. Both generator arms are optimized through the drafter loss during adaptation.

Observed behavior. The pretrained generator starts at a lower validation loss than the fresh generator. By the end of the displayed adaptation, the two generator validation curves meet at a similar loss. The transferred ordinary LoRA changes little, while the fresh ordinary LoRA improves but remains above the generators. Figure 2 therefore shows an initialization benefit from bank warmup without an observed final-loss advantage at this budget.

Design decision. We retired the LoRA-bank approach as unhelpful for the current workflow. We use the original examples directly to train the generator with the drafting loss. Bank reconstruction first turns those examples into trained LoRA factors and then asks the generator to reproduce that proxy. Direct training optimizes the desired drafting objective itself and removes both bank construction and proxy reconstruction from the workflow. The observed fresh-generator catch-up supports choosing this simpler route; no further bank experiments are part of the current plan.

Scope of the claim. Direct training is the chosen, more useful approach for this project, rather than a demonstrated universal ordering of the methods. This adaptation study is not a matched-total-budget comparison of using the same examples for bank warmup versus direct pretraining. It also mixes conditioning and parameter-capacity differences between generators and ordinary LoRA. Raw metrics, repeated seeds, and complete historical provenance are missing. We retain the new screenshot without reusing approximate endpoint values from older screenshots in the working notes.

4.3. GEN-003: Current Four-Model Throughput Record

Status: Measured; supplied summary, incomplete provenance.

Evidence: Suite `comparison-v2`; received 2026-10-22. Execution date not supplied. Tables are generated from the frozen JSON transcription.

Question. What throughput values were recorded for the four model variants on the selected in-distribution and out-of-distribution categories?

Setup. The source is the suite owner's supplied `summary.md`. Tables 1 and 2 preserve all 28 reported measurements without adding older model rows. Step counts follow the supplied labels. Correctness was not checked. This record does not include the native per-request outputs, latency statistics, acceptance counters, or server provenance for this suite.

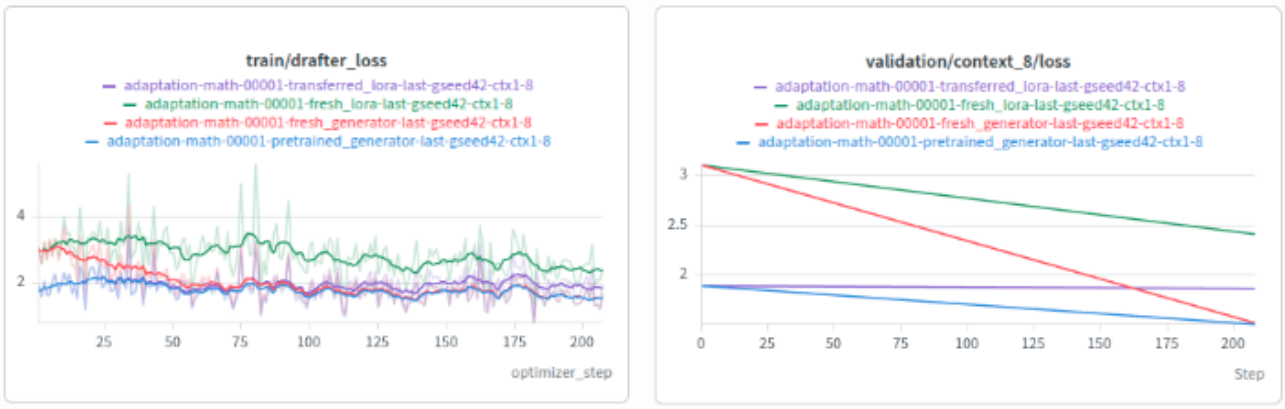


Figure 2. GEN-004: supplied adaptation curves for pretrained/fresh generators and transferred/fresh ordinary LoRAs. Bank pretraining improves the starting point, but the fresh generator reaches a similar final context-8 validation loss. The displayed comparison does not match total pretraining-plus-adaptation cost.

Table 1. In-distribution Nemotron: recorded aggregate output tokens/s. Correctness was not checked; step counts follow supplied labels.

Model	Math	Code	STEM	Chat
EAGLE3 base	64.34	60.93	59.28	54.97
Trained LoRA	72.50	64.79	64.23	56.35
Generator, 2K steps	77.58	68.11	66.50	57.58
Generator, 4K steps	79.11	69.83	67.92	56.49

Table 2. Out-of-distribution Red Hat: recorded aggregate output tokens/s. Correctness was not checked; step counts follow supplied labels.

Model	HumanEval	Math reasoning	Summarization
EAGLE3 base	68.32	66.55	58.29
Trained LoRA	67.24	67.19	58.30
Generator, 2K steps	69.24	67.99	58.81
Generator, 4K steps	69.44	68.32	58.69

Record notes. The values are recorded separately by category and checkpoint. We do not pool categories into an overall score or infer a final model ranking. The checkpoint pair provides observations at two reported training stages, not a repeated or controlled causal estimate of additional training. The two tables do not establish output quality or a target-only decoding speedup.

Missing evidence. Native GuideLLM results, effective configurations, hardware metadata, prompt fingerprints, checkpoint hashes, and repeated measurements remain to be attached to this suite record.

4.4. SYS-001: Cold and Warm Activation Reuse

Status: Measured; console timings transcribed in the cache record.

Evidence: Pilot logs received 2026-10-16; reported A100 80 GB, 400 validation payloads evaluated in context-4 groups.

Question. What stage wall times were observed before and after activation payloads were available in the shared RAM cache?

Table 3. Validation wall seconds from the cache pilot. The first pass includes extraction and startup effects.

Category	First pass (s)	Warm pass (s)
Math	35.18	6.25
Code	24.49	5.47
STEM	15.53	3.41
Chat	21.70	4.93
All categories	96.90	20.06

Record notes. The payloads occupied 16.81 GiB. Subsequent jobs in the same process reported initial validation passes of 19.99 and 16.86 seconds. The first pass includes extraction and startup effects, whereas the warm pass still performs generator/drafter evaluation. The difference is an end-to-end observation, not an isolated cache microbenchmark or inference speedup. In a direct-training pilot the cache reached 63.97 GiB, with 365 evictions and 69.67 GiB final process RSS. Later prefetch and summary-cache changes have no separate GPU timing comparison in the evidence included here.

Missing evidence. Repeated cold/warm trials with separated extraction, construction, transfer, and compute timings remain available follow-up work.

4.5. GEN-002: Output Repeatability Investigation

Status: Investigation deferred; exact sequence comparisons recorded.

Evidence: Local correctness reports summarized in the Markdown registry, observation dated 2026-10-21. Raw reports are not archived with this report.

Question. Were target-only and speculative outputs repeatable under the supplied prompt/settings fingerprints?

On 50 Nemotron math prompts, original versus repeated target-only decoding matched exactly for 23/50 sequences; original versus repeated EAGLE3 decoding matched for 50/50. Repeated target-only versus repeated EAGLE3 matched for 17/50. These are whole-sequence match counts, not token-error rates.

Record notes. The cause remains unresolved. Runtime environment, numerical behavior, and decoding implementation differences have not been isolated. Current timing experiments explicitly record correctness as unchecked or unverified. No correctness claim is inferred from the throughput tables.

Missing evidence. Preserve and compare complete runtime provenance and repeat a controlled reference experiment before making an exact-output claim.

4.6. BATCH-001: Shared-Adapter Concurrency Sweep

Status: Implemented; GPU measurements pending.

Evidence: Implementation revision `c0ee0e4`; CPU lifecycle and HTTP tests, plus an audit of the pinned vLLM hook upgrade.

Question. What measurements arise when a generator conditions one shared adapter on a fixed group at concurrency 1, 2, 4, or 8?

Planned measurement configuration. Reuse the frozen category prompts for all model/concurrency variants. Record generated output tokens/s, speculative acceptance where available, request latency, actual mean batch size, and generator/merge/restoration costs. Keep the adapter fixed while a group drains. Original-drafter and static-LoRA variants use matching closed groups.

Available checks. Tests cover packed prompt boundaries, one generator invocation per group, restoration after the last member, delayed HTTP cleanup, closed admission, partial groups, and suite resume across concurrency variants. The initial broader CPU/HTTP run passed 94 tests; a later focused run after controller refinements passed 50. These runs overlap and are not additive.

Pending evidence. No A100 execution, batched throughput table, acceptance comparison, or memory measurement is supplied. CPU checks establish neither GPU compatibility under load nor a performance benefit.

5. Implementation and Investigation Notes

5.1. Activation Supply and Cache Lifetime

Training consumes local saved trajectories, with activations constructed on demand by an external vLLM extraction server. A bounded CPU cache retains full tensors, while prompt summaries can be reused separately. Cache capacity is not a cap on process RSS: temporary construction buffers, model state, and other allocations contribute additional memory. Persistent summaries, prefetching, and concurrency controls are implementation features; only the pilot timings in SYS-001 are treated as measurements here.

5.2. Checkpoint Publication and Recovery

The full-data training workflow supports local recovery and Hub snapshots containing generator weights, optimizer state, progress, RNG state, resolved configuration, and available provenance. Inference bundles can be downloaded without optimizer checkpoints. Recovery capability is recorded as workflow support; it does not demonstrate completion of the ten-epoch training plan.

5.3. Independent Serving and Benchmarking

The suite separates model serving from the GuideLLM client. It launches and cleans up owned server/client process groups, exports category workloads once, records attempt state, and can skip completed measurements with matching artifact checksums. Model type selects an original drafter, static-LoRA bundle, or generator bundle. Throughput can be measured without an additional correctness-generation pass; the absence of that pass is recorded explicitly.

5.4. Fixed-Batch Integration Boundaries

The new scheduler leaves vacant slots empty until a group finishes. It permits a partial group after a 50 ms collection window in the example configuration. The runtime rejects KV preemption or incomplete initial group admission rather than reuse draft cache entries under a different generated adapter. Larger concurrency also increases the prefill/profile token budget and may change GPU memory requirements. These are implementation constraints awaiting the GPU experiment in BATCH-001.

Collection, queueing, uneven request completion, and partial tails are part of the measured serving protocol. Closed groups are not equivalent to unrestricted continuous batching. Batch membership may vary with arrival/completion order, even when the same prompt list is reused across

model variants.

6. Open Questions and Planned Evidence

This section is a work queue rather than a conclusion. Entries are retained or revised when new evidence arrives.

1. **Suite provenance.** Attach the native artifacts for `comparison-v2`; resolve exact checkpoint progress, hardware, concurrency, prompt identities, and effective serving settings.
2. **Repetition.** Repeat matched serving measurements to record uncertainty rather than interpret small single-run differences.
3. **Batching.** Run BATCH-001 on the intended GPU. Check actual batch sizes, memory, request completion/restoration, and acceptance before treating it as a completed performance experiment.
4. **Reference behavior.** Continue GEN-002 when requested, separating target-only repeatability from adapter-related behavior.
5. **Scaling interpretation.** Distinguish the fixed-update result from matched-epoch training and broader validation. GEN-001 has no repeated seeds or uncertainty estimates. Bank warmup is retired; GEN-004 documents the decision rather than scheduling further bank experiments.
6. **Training progression.** Extend verified checkpoint observations as snapshots become available. Do not infer later training outcomes from the planned epoch schedule.
7. **Systems costs.** Separate activation extraction, preprocessing, transfer, and training compute when a further timing study is commissioned.

Future report versions should add measurements under the relevant stable identifier, or create a new identifier if the comparison protocol changes. Missing measurements remain missing; implementation status is not substituted for an experimental result.

A. Evidence Registry and Reproducibility

A.1. Source Registry

Repository paths below identify the working records. The report's build manifest records hashes of the source files available when this PDF was built.

GEN-001 `docs/experiments/results/GEN-001-direct-data-scaling.md`; scaling screenshot copied unchanged into `reporting/assets/`.

GEN-004 `docs/experiments/results/GEN-004-bank-warmup-adaptation.md`; adaptation screenshot copied unchanged into `reporting/assets/`.

Architecture `src/speculators/generator/model.py` and `src/speculators/generator/config.py`, included in the source snapshot.

GEN-003 `docs/experiments/results/GEN-003-generator-suite-throughput.md`; the copied numeric source is `reporting/data/comparison-v2.json`.

SYS-001 `docs/experiments/eagle3_activation_ram_cache.md`; the copied timing source is `reporting/data/cache-pilot.json`.

GEN-002 `docs/experiments/results/GEN-002-http-token-equality.md`.

BATCH-001 `docs/experiments/eagle3_benchmark_suite.md` and `docs/experiments/eagle3_generator_http.md`; implementation revision `c0ee0e4b7c59ff8ff6ee8c201e591dceb0589635`.

Training `docs/experiments/eagle3_generator.md`, `docs/experiments/eagle3_generator_campaign_online.md`, and `docs/experiments/eagle3_generator_full_data.md`.

Datasets `docs/experiments/speculator_benchmarks_inventory.md`.

Historical training ablations appear separately from the four-model serving comparison. Screenshots preserve qualitative evidence without inventing raw metric exports. Bank reconstruction is retained as a historical decision record, not as an active experiment. The supplied adaptation image has a different endpoint scale from the older approximate transcription; numerical readings from these records are not pooled.

A.2. Reference Revisions and Artifact Requirements

The documented target reference is Qwen/Qwen3-8B at `b968826d9c46dd6066d109eabc6255188de91218`; the original drafter reference is RedHatAI/Qwen3-8B-speculator.eagle3 at `08610ffa01dd9f16731fe8f627b85905b6aa51c4`. The inspected Red Hat dataset revision is `2ae86affa2cb97a972b7fc681dd51c04fbff083e`. These reference revisions do not replace missing per-run checkpoint hashes.

Publishing an experiment requires its available training/evaluation/serving commands, resolved configurations, model hashes, package/runtime details, and source patches. Native GuideLLM outputs and workload manifests should accompany performance records. This report contains a summary-level transcription for GEN-003; it does not claim to include those missing run artifacts.

A.3. Commands and Manual Update Policy

The shared-batch experiment is launched after updating the pinned vLLM hook:

```
python \
  scripts/apply_vllm_generator_patch.py \
  --apply
BENCH_CONFIG=examples/eagle3-lora/\
benchmark_suite.yaml
speculators benchmark suite \
  --config "$BENCH_CONFIG" \
  --output-root output/shared-batch-v1
```

These are workflow commands, not a claim that this report's throughput tables were produced by that new batch configuration. Actual launch commands must be read from the associated run provenance. Use a persistent output directory on Colab. Every server launch must pass `--provenance-dir`; the suite supplies this automatically.

Report updates occur only on explicit request. Markdown remains the working experiment record. Before extending a report, freeze new input measurements, update statuses and evidence dates, preserve earlier snapshots, and regenerate tables from their numeric sources. New compiler build timestamps do not change the evidence cutoff.

B. Report Change History

Version 01, 2026-10-22. Initial experimental record: current four-model in-/out-of-distribution tables, cache pilot timings, deferred repeatability investigation, and pending shared-batch GPU measurements. No final conclusions are asserted.

Version 02, 2026-10-23. Retitled *Dynamic Speculators*; expanded the generator architecture; added fixed-update data scaling with the ordinary LoRA baseline and the adaptation figure; recorded the decision to retire bank warmup in favor of direct drafting-loss training. Version 01 is preserved.